

GEMM File: check_ppcg_pluto_mar_2026/ppcg_outputs/gemm_ppcg_openmp.c

/**

* This version is stamped on May 10, 2016

*

* Contact:

* Louis-Noel Pouchet <pouchet.ohio-state.edu>

* Tomofumi Yuki <tomofumi.yuki.fr>

*

* Web address: <http://polybench.sourceforge.net>

*/

/* gemm.c: this file is part of PolyBench/C */

#include <stdio.h>

#include <unistd.h>

#include <string.h>

#include <math.h>

/* Include polybench common header. */

#include <polybench.h>

/* Include benchmark-specific header. */

#include "gemm.h"

/* Array initialization. */

static

void init_array(int ni, int nj, int nk,

DATA_TYPE *alpha,

DATA_TYPE *beta,

DATA_TYPE POLYBENCH_2D(C,NI,NJ,ni,nj),

DATA_TYPE POLYBENCH_2D(A,NI,NK,ni,nk),

DATA_TYPE POLYBENCH_2D(B,NK,NJ,nk,nj))

{

int i, j;

*alpha = 1.5;

*beta = 1.2;

for (i = 0; i < ni; i++)

for (j = 0; j < nj; j++)

C[i][j] = (DATA_TYPE) ((i*j+1) % ni) / ni;

for (i = 0; i < ni; i++)

for (j = 0; j < nk; j++)

A[i][j] = (DATA_TYPE) (i*(j+1) % nk) / nk;

for (i = 0; i < nk; i++)

```

    for (j = 0; j < nj; j++)
        B[i][j] = (DATA_TYPE) (i*(j+2) % nj) / nj;
}

```

/* DCE code. Must scan the entire live-out data.

Can be used also to check the correctness of the output. */

static

```

void print_array(int ni, int nj,
                DATA_TYPE POLYBENCH_2D(C,NI,NJ,ni,nj))
{
    int i, j;

    POLYBENCH_DUMP_START;
    POLYBENCH_DUMP_BEGIN("C");
    for (i = 0; i < ni; i++)
        for (j = 0; j < nj; j++) {
            if ((i * ni + j) % 20 == 0) fprintf (POLYBENCH_DUMP_TARGET, "\n");
            fprintf (POLYBENCH_DUMP_TARGET, DATA_PRINTF_MODIFIER, C[i][j]);
        }
    POLYBENCH_DUMP_END("C");
    POLYBENCH_DUMP_FINISH;
}

```

/* Main computational kernel. The whole function will be timed,
including the call and return. */

static

```

void kernel_gemm(int ni, int nj, int nk,
                DATA_TYPE alpha,
                DATA_TYPE beta,
                DATA_TYPE POLYBENCH_2D(C,NI,NJ,ni,nj),
                DATA_TYPE POLYBENCH_2D(A,NI,NK,ni,nk),
                DATA_TYPE POLYBENCH_2D(B,NK,NJ,nk,nj))
{
    int i, j, k;

    //BLAS PARAMS
    //TRANSA = 'N'
    //TRANSB = 'N'
    // => Form C := alpha*A*B + beta*C,
    //A is NlXNK
    //B is NKxNJ
    //C is NlXNJ

```

```

/* ppcg generated CPU code */

#pragma omp parallel for
for (int c0 = 0; c0 < ni; c0 += 1)
    for (int c1 = 0; c1 < nj; c1 += 1) {
        C[c0][c1] *= beta;
        for (int c2 = 0; c2 < nk; c2 += 1)
            C[c0][c1] += ((alpha * A[c0][c2]) * B[c2][c1]);
    }
}

int main(int argc, char** argv)
{
    /* Retrieve problem size. */
    int ni = NI;
    int nj = NJ;
    int nk = NK;

    /* Variable declaration/allocation. */
    DATA_TYPE alpha;
    DATA_TYPE beta;
    POLYBENCH_2D_ARRAY_DECL(C, DATA_TYPE, NI, NJ, ni, nj);
    POLYBENCH_2D_ARRAY_DECL(A, DATA_TYPE, NI, NK, ni, nk);
    POLYBENCH_2D_ARRAY_DECL(B, DATA_TYPE, NK, NJ, nk, nj);

    /* Initialize array(s). */
    init_array (ni, nj, nk, &alpha, &beta,
                POLYBENCH_ARRAY(C),
                POLYBENCH_ARRAY(A),
                POLYBENCH_ARRAY(B));

    /* Start timer. */
    polybench_start_instruments;

    /* Run kernel. */
    kernel_gemm (ni, nj, nk,
                 alpha, beta,
                 POLYBENCH_ARRAY(C),
                 POLYBENCH_ARRAY(A),
                 POLYBENCH_ARRAY(B));

    /* Stop and print timer. */

```

```
polybench_stop_instruments;  
polybench_print_instruments;
```

```
/* Prevent dead-code elimination. All live-out data must be printed
   by the function call in argument. */
polybench_prevent_dce(print_array(ni, nj, POLYBENCH_ARRAY(C)));
```

```
/* Be clean. */
POLYBENCH_FREE_ARRAY(C);
POLYBENCH_FREE_ARRAY(A);
POLYBENCH_FREE_ARRAY(B);
```

```
return 0;
```

}

Jacobi-2D File: check_ppcg_pluto_mar_2026/ppcg_outputs/jacobi-2d_ppcg_openmp.c:

/ **

* This version is stamped on May 10, 2016

*

* Contact:

* Louis-Noel Pouchet <pouchet.ohio-state.edu>

* Tomofumi Yuki <tomofumi.yuki.fr>

*

* Web address: <http://polybench.sourceforge.net>

*/

```
/* jacobi-2d.c: this file is part of PolyBench/C */
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
/* Include polybench common header. */
```

```
#include <polybench.h>
```

```
/* Include benchmark-specific header. */
```

```
#include "jacobi-2d.h"
```

```
/* Array initialization. */
```

static

```
void init_array (int n,
```

DATA TYPE POLYBENCH 2D(A,N,N,n,n),

DATA_TYPE POLYBENCH_2D(B,N,N,n,n))

{

```

int i, j;

for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
    {
        A[i][j] = ((DATA_TYPE) i*(j+2) + 2) / n;
        B[i][j] = ((DATA_TYPE) i*(j+3) + 3) / n;
    }
}

/* DCE code. Must scan the entire live-out data.
   Can be used also to check the correctness of the output. */
static
void print_array(int n,
                 DATA_TYPE POLYBENCH_2D(A,N,N,n,n))

{
    int i, j;

    POLYBENCH_DUMP_START;
    POLYBENCH_DUMP_BEGIN("A");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            if ((i * n + j) % 20 == 0) fprintf(POLYBENCH_DUMP_TARGET, "\n");
            fprintf(POLYBENCH_DUMP_TARGET, DATA_PRINTF_MODIFIER, A[i][j]);
        }
    POLYBENCH_DUMP_END("A");
    POLYBENCH_DUMP_FINISH;
}

/* Main computational kernel. The whole function will be timed,
   including the call and return. */
static
void kernel_jacobi_2d(int tsteps,
                     int n,
                     DATA_TYPE POLYBENCH_2D(A,N,N,n,n),
                     DATA_TYPE POLYBENCH_2D(B,N,N,n,n))
{
    int t, i, j;

    /* ppcg generated CPU code */

```

```

#define ppcg_min(x,y)  ({ __typeof__(x) _x = (x); __typeof__(y) _y = (y); _x < _y ? _x : _y; })
#define ppcg_max(x,y)  ({ __typeof__(x) _x = (x); __typeof__(y) _y = (y); _x > _y ? _x : _y; })
for (int c0 = 0; c0 < tsteps; c0 += 1)
    for (int c1 = 2 * c0 + 1; c1 < n + 2 * c0; c1 += 1)
        for (int c2 = ppcg_max(2 * c0 + 1, -n + c1 + 3); c2 < ppcg_min(n + 2 * c0, n + c1 - 2); c2 +=
1) {
            if (c1 >= 2 * c0 + 2 && c2 >= 2 * c0 + 2)
                A[-2 * c0 + c1 - 1][-2 * c0 + c2 - 1] = (0.200000000000000001 * (((B[-2 * c0 + c1 - 1][-2 *
c0 + c2 - 1] + B[-2 * c0 + c1 - 1][-2 * c0 + c2 - 2]) + B[-2 * c0 + c1 - 1][-2 * c0 + c2]) + B[-2 * c0 +
c1][-2 * c0 + c2 - 1]) + B[-2 * c0 + c1 - 2][-2 * c0 + c2 - 1]));
            if (n + 2 * c0 >= c1 + 2 && n + 2 * c0 >= c2 + 2)
                B[-2 * c0 + c1][-2 * c0 + c2] = (0.200000000000000001 * (((A[-2 * c0 + c1][-2 * c0 + c2] +
A[-2 * c0 + c1][-2 * c0 + c2 - 1]) + A[-2 * c0 + c1][-2 * c0 + c2 + 1]) + A[-2 * c0 + c1 + 1][-2 * c0 +
c2]) + A[-2 * c0 + c1 - 1][-2 * c0 + c2]));
        }
}

```

```

int main(int argc, char** argv)
{
    /* Retrieve problem size. */
    int n = N;
    int tsteps = TSTEPS;

    /* Variable declaration/allocation. */
    POLYBENCH_2D_ARRAY_DECL(A, DATA_TYPE, N, N, n, n);
    POLYBENCH_2D_ARRAY_DECL(B, DATA_TYPE, N, N, n, n);

    /* Initialize array(s). */
    init_array (n, POLYBENCH_ARRAY(A), POLYBENCH_ARRAY(B));

    /* Start timer. */
    polybench_start_instruments;

    /* Run kernel. */
    kernel_jacobi_2d(tsteps, n, POLYBENCH_ARRAY(A), POLYBENCH_ARRAY(B));

    /* Stop and print timer. */
    polybench_stop_instruments;
    polybench_print_instruments;

    /* Prevent dead-code elimination. All live-out data must be printed
    by the function call in argument. */
}

```

```

polybench_prevent_dce(print_array(n, POLYBENCH_ARRAY(A)));

/* Be clean. */
POLYBENCH_FREE_ARRAY(A);
POLYBENCH_FREE_ARRAY(B);

return 0;
}
Cholesky File: check_ppcg_pluto_mar_2026/ppcg_outputs/cholesky_ppcg_openmp.c:
#include <omp.h>
#include <math.h>
#define ceild(n,d) ceil(((double)(n))/((double)(d)))
#define floord(n,d) floor(((double)(n))/((double)(d)))
#define max(x,y) ((x) > (y)? (x) : (y))
#define min(x,y) ((x) < (y)? (x) : (y))

/**
 * This version is stamped on May 10, 2016
 *
 * Contact:
 *   Louis-Noel Pouchet <pouchet.ohio-state.edu>
 *   Tomofumi Yuki <tomofumi.yuki.fr>
 *
 * Web address: http://polybench.sourceforge.net
 */
/* cholesky.c: this file is part of PolyBench/C */

#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <math.h>

/* Include polybench common header. */
#include <polybench.h>

/* Include benchmark-specific header. */
#include "cholesky.h"

/* Array initialization. */
static
void init_array(int n,
                DATA_TYPE POLYBENCH_2D(A,N,N,n,n))
{

```

```

int i, j;

for (i = 0; i < n; i++)
{
    for (j = 0; j <= i; j++)
        A[i][j] = (DATA_TYPE)(-j % n) / n + 1;
    for (j = i+1; j < n; j++) {
        A[i][j] = 0;
    }
    A[i][i] = 1;
}

/* Make the matrix positive semi-definite. */
int r,s,t;
POLYBENCH_2D_ARRAY_DECL(B, DATA_TYPE, N, N, n, n);
for (r = 0; r < n; ++r)
    for (s = 0; s < n; ++s)
        (POLYBENCH_ARRAY(B))[r][s] = 0;
for (t = 0; t < n; ++t)
    for (r = 0; r < n; ++r)
        for (s = 0; s < n; ++s)
            (POLYBENCH_ARRAY(B))[r][s] += A[r][t] * A[s][t];
for (r = 0; r < n; ++r)
    for (s = 0; s < n; ++s)
        A[r][s] = (POLYBENCH_ARRAY(B))[r][s];
POLYBENCH_FREE_ARRAY(B);

}

/* DCE code. Must scan the entire live-out data.
   Can be used also to check the correctness of the output. */
static
void print_array(int n,
                 DATA_TYPE POLYBENCH_2D(A,N,N,n,n))

{
    int i, j;

    POLYBENCH_DUMP_START;
    POLYBENCH_DUMP_BEGIN("A");
    for (i = 0; i < n; i++)
        for (j = 0; j <= i; j++) {
            if ((i * n + j) % 20 == 0) fprintf (POLYBENCH_DUMP_TARGET, "\n");

```

```

    fprintf (POLYBENCH_DUMP_TARGET, DATA_PRINTF_MODIFIER, A[i][j]);
}
POLYBENCH_DUMP_END("A");
POLYBENCH_DUMP_FINISH;
}

```

```

/* Main computational kernel. The whole function will be timed,
   including the call and return. */

```

```
static
```

```
void kernel_cholesky(int n,
                    DATA_TYPE POLYBENCH_2D(A,N,N,n,n))
{
    int i, j, k;

```

```
/* ppcg generated CPU code */
```

```

for (int c0 = 0; c0 < n; c0 += 1) {
    A[c0][c0] = sqrt(A[c0][c0]);
    for (int c1 = c0 + 1; c1 < n; c1 += 1) {
        A[c1][c0] /= A[c0][c0];
        #pragma omp parallel for
        for (int c2 = c0 + 1; c2 < c1; c2 += 1)
            A[c1][c2] -= (A[c1][c0] * A[c2][c0]);
        A[c1][c1] -= (A[c1][c0] * A[c1][c0]);
    }
}
}

```

```
int main(int argc, char** argv)
```

```

{
    /* Retrieve problem size. */
    int n = N;

```

```
/* Variable declaration/allocation. */
```

```
POLYBENCH_2D_ARRAY_DECL(A, DATA_TYPE, N, N, n, n);
```

```
/* Initialize array(s). */
```

```
init_array (n, POLYBENCH_ARRAY(A));
```

```
/* Start timer. */
```

```
polybench_start_instruments;

/* Run kernel. */
kernel_cholesky (n, POLYBENCH_ARRAY(A));

/* Stop and print timer. */
polybench_stop_instruments;
polybench_print_instruments;

/* Prevent dead-code elimination. All live-out data must be printed
   by the function call in argument. */
polybench_prevent_dce(print_array(n, POLYBENCH_ARRAY(A)));

/* Be clean. */
POLYBENCH_FREE_ARRAY(A);

return 0;
}
```